01 / PRODUCT

Two editions from one set of checked stories.

Wire tracks what frontier labs and the people inside them publish, drops the noise, and writes a weekly edition for each of two teams at a private asset manager: investment and AI.

WHAT IT IS FOR

Tell each team what changed, and show the evidence.

330 items collected → 180 passed the filter → 34 read in full → 9 stories → 5 + 2 AI + investment alerts

The first live run, 17–24 Sep 2026. Code dropped 109 replies, 40 prereleases and a duplicate before any model read them.

Built by Dinobridge for a private asset manager, September 2026. The code lives in a private repository.

INVESTMENT

Does this change a thesis?

For portfolio managers and analysts. The writer knows the positions the customer shared with us, where each sits in the AI supply chain, and says what would show a mechanism wrong.

AI TEAM

What should we test?

For the team building the customer's own AI platform. Say what is worth adopting, how to test it, and what would show it does not help.

WHAT IT FOUND

INVESTMENT · ACT

OpenAI's 8 GW Ohio site runs only NVIDIA

NVIDIA also invests in the developer, and the first capacity has a delivery date. Filed as dated NVIDIA demand.

AI · ACT / INVESTMENT · WATCH

DeepSeek cuts memory per token to a quarter

V4.1-Flash needs a quarter of the HBM per token of its predecessor, with open weights. Filed against the memory-per-token thesis behind Micron.

AI TEAM

Passing tests is not serving

In NVIDIA's benchmark, 69% of agent patches pass the tests but only 46% survive live serving. Worth testing in the team's own evals.

INVESTMENT · WATCH

Dean and Ghemawat leave Google to found a lab

Filed under new labs, with Alphabet as founding investor. I could not date a stock reaction to a departure, so it is not a price signal.

FROM BUILDING IT

A position total is not exposure

Part of the shared total is option notional: one position, Amazon, looks about 2.5 times its equity size. The editions quote equity and options separately.

02 / SYSTEM CONTEXT

A static app, a database, and one agent session per run.

The app reads the database directly. Each run is one managed agent session that fetches sources and writes its results back.



NO BACKEND

The app reads the database directly

Sign-in and access rules decide what each member sees. There is no server of mine to run or patch.

ONE SESSION PER RUN

Every run is pinned

Run control starts a managed session with fixed prompt and code versions and a spending cap. Any edition traces to exactly what wrote it.

TWO CLOCKS

Weekly judgment, daily checks

The judged run starts every Monday. A code-only job checks tracked pages every morning and can alert between editions.

Trade-off With no backend, the database access rules are the whole security model, so they have their own tests.

03 / DATA FLOW

One weekly run, start to finish.

Code handles the repeatable steps, Haiku reads what the filter keeps, and one Opus agent decides what matters.



WHO IS KEY

The layer below the leaders

Labs through their own channels, and the researchers whose papers and code come first. Leaders already speak through the lab. Each person has a stated reason; ex-lab startups are labs of their own.

ONE PERSON, MANY ACCOUNTS

Linked only with proof

An X handle, GitHub account or arXiv name joins a person only if their own page shows it, never by name. Official, personal and press sources are labelled apart.

COVERAGE

Few sources, done well

Lab channels, arXiv, GitHub, Hugging Face, talks, and X through Apify. LinkedIn and anything behind a login are out. A source that fails twice or goes silent is flagged.

NOISE

Dropped by shape, in code

Replies, reposts, duplicates and stale posts go before any model reads them, each with a reason a curator can overrule. There is no topic rule, so an unexpected subject still gets through.

One card per item Every kept item is read once into the same short shape: what it says, who and what it names, and its claims, each with an exact quote. Everything after works from cards: the analyst scans and searches them, links them to the register and to past stories, and cites their quotes as evidence.

Rejected A model triaging every item instead of the filter: it recreates the reading cost the filter exists to cut.

04 / MODEL CHOICES

An agent only where the next step depends on what it reads.

Three agents in total. Anything with a right answer runs as code.

CODE

Anything with a right answer

- Fetching and filtering
- Quotes and numbers checked
- Each edition checked
- Daily change alerts

Cheap, exact and testable.

ONE MODEL CALL

Reading at scale

- Haiku summarises every kept item
- Sonnet reads long documents
- Haiku checks quote support
- Opus ranks the stories

Sonnet summaries moved the picks no more than a second rater did, so Haiku reads first.

AGENT

Decisions that branch

- Opus decides what matters
- It chooses when to read deeper
- It sends a scout for a source
- Sonnet writers compose, then review

The strongest model sits where a wrong call costs most.

WHAT COUNTS AS MATERIAL

Every story answers a question.

Investment: inference cost, memory, compute, power, new labs, moats.
AI team: evaluation, open models, pipelines, tooling.
Plus neutral triggers, and at most two “unexpected” a week.

ACT	High score Strong evidence
WATCH	Good score Moderate evidence

THE THREE AGENTS

ANALYST · OPUS 5.5

Coordinates the run

Runs the pipeline’s own tools in its sandbox, reads cards and past weeks, and stores the stories. The only agent that can call the other two.

SCOUT · SONNET 5

Finds one primary source

Web search and fetch only, with no write access. It answers one question from the analyst and returns the source.

WRITER · SONNET 5

Writes one edition

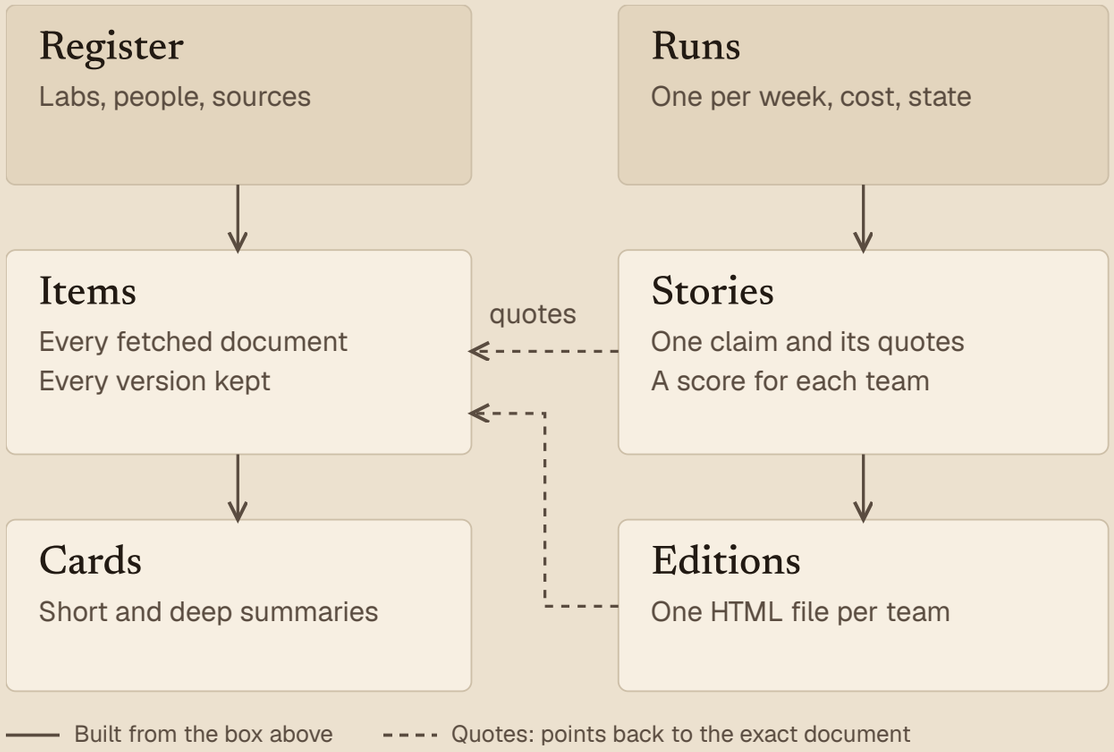
Gets one team’s stored facts and the report skill, and writes the HTML. Called again with the rubric, it reviews both editions.

When a step fails	A failed summary is retried once. A story that fails its checks is fixed once, then cut. A failed ranking leaves scores marked provisional. A claim with no primary source sends a scout.
Rejected	An agent per document: much of its reading went on items later dropped. No agent at all: the next step depends on what was just read.

05 / DATA MODEL & THE APP

One story per fact, with a view for each team.

A story points at the exact version of every page it quotes, and a person gets credit only for what they publish themselves.



ONE STORY, TWO VIEWS · FROM THE REPLAY

DeepSeek V4.1-Flash: its KV cache needs ¼ the HBM and ⅛ the SSD

One claim and eleven quotes from five source items. Each team reads it through its own question:

INVESTMENT · WATCH	Evidence against memory-per-token growth at Micron; total demand can still rise with volume.
AI TEAM · ACT	A week's test as a cheaper backend for extraction and tool use. It fails if task success drops, or the saving does not show in cost per task.

Rejected: separate stories for each team. With one story and a view for each team, the two editions cannot disagree on a fact.

WHO CAN DO WHAT

ROLE	CAN
Reader	Read editions, stories and the noise list Vote and comment
Curator	Everything a reader can Edit the register, start or resume a run
Pipeline	Write runs, stories and editions Cannot change members or finished editions

CONTRIBUTORS

Credit for their own work

A person earns credit only when an alerting story cites their own work about their own lab. Counting every mention instead put two outside commentators above every lab researcher.

TREND

Busier or quieter than before

Each lab and person page compares how many stories they produced in the last 30 days with the 60 days before, and labels it more active, less active, steady, or new.

THE APP

Read, trace, correct

Readers see today's alerts and both editions, and can trace any story to its sources and why it was flagged. Curators also restore dropped items and edit the labs and people.

06 / QUALITY & GROUNDING

What I measured, and what it changed.

Every label is a separate model's, given blind, and every figure has its sample size. Checks in code block fabricated claims in every run.

How I evaluate Outside the product, in Claude Code. A separate model that no pipeline stage uses labels samples, judges picks blind, and compares the old and new versions week by week over the replay. Its labels never reach the pipeline. The repository's evaluation record gives each figure with its 95% interval.

THE FILTER

Does it keep what matters?

30/30 items the labeller kept, the filter kept too

An audit of 106 dropped items found 7 worth keeping, a system card among them, and two rules changed. Precision is 0.57: the analyst drops the rest.

THE SUMMARIES

Is the cheap model enough?

19/24 picks unchanged on Sonnet's cards, the same as a second rater on Haiku's

Sonnet wrote the best long summary in 12 of 20, but it did not change the picks. Haiku reads first; Sonnet reads deep on request.

THE CHECKS

Do false stories get through?

20/20 planted false stories refused, and the clean control passes

The set found a gap: a number with a unit, like "2M context", skipped the number check. Fixed.

THE SCORE

Does it mean something?

22/24 of the analyst's top picks rated useful; 15 were in the rater's own top 3

The judge preferred the old version's alerts 28 times to 19. After recalibration, 17 of those 28 now alert.

WHAT GETS CHECKED IN EVERY RUN

01 **Before a story is stored**

Every quote and number is found in a source, and at least one quote supports the claim. A match shows the source said it, not that it is true.

02 **Before an edition is released**

Every citation and figure traces to a stored story; nothing loads from outside.

03 **Evidence has a ceiling**

Press coverage alone is weak evidence, whatever the analyst thinks of it.

GROUND TRUTH

Model labels now, the teams' labels next.

I could not label by hand in five days, so a separate model labels blind. That shows the system is consistent, not that it is right, and the labeller shares a model family with the pipeline.

One outside check failed: the contributor score against OpenAlex output, Spearman −0.27 over 15 people. OpenAlex counts publishing, not originating, so it was the wrong check. The golden set on page 09 replaces the model labels with the teams' own judgment.

07 / RELIABILITY & SECURITY

Failures are recorded, and the run closes partial.

Progress and source health live in the database, outside the session, so a failed run can be resumed where it stopped.

WHEN SOMETHING FAILS

FAILURE	WHAT HAPPENS
A source fails	Recorded; the run closes **partial** and the edition names the gap
The session idles or runs out of budget	A watchdog closes the run within minutes; a hung session is closed at a hard time ceiling
A curator resumes a run	It restarts at the last finished stage, within the remaining budget
An edition fails its checks	It is not released
The Claude platform is down	The run is recorded as failed; a curator resumes it when the platform is back. Page 10 adds a second provider

ACCESS

Readers are not operators.

Invite-only sign-in, reader and curator roles, and database guards on what the pipeline may change. Fetched pages are read by tool-less model calls and marked untrusted.

GENERATED HTML

Editions run scripts but carry no sign-in.

They run sandboxed, cannot load anything from outside, and never see the reader's session.

What I watch A cost for every run, and a trace once its session is imported. Failed runs and newly failing sources raise an operator alert in the app, ready to post to Slack.

Rejected My own workers and queue on the plain API: I would own every loop, retry and tool call. A workflow framework would be a second orchestrator.

08 / COST & HOW I WORKED

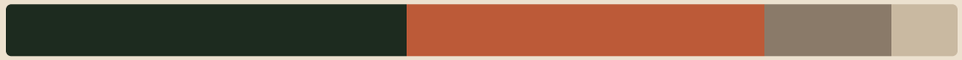
What a run costs, and the loop that built it.

The analyst and the writers are 79% of a $12.52 run. Four replays decided what stayed.

WHERE THE MONEY GOES

$12.52

One weekly run at list prices, 68 minutes. The session is capped at $15.



Analyst, Opus	$5.24	Writers and review, Sonnet	$4.65
Cards, ranking, support checks	$1.66	Scouts, Sonnet	$0.88

WORKFLOW	COST
Daily change check	No model cost; 42 seconds on GitHub Actions
Replay of 12 weeks	$14.15 of API; judging ran as Claude Code subagents on the subscription
Whole build	About €100 on the Anthropic API and a €19 Apify plan for X: a little over the €100 budget

Tokens per weekly run: 17.6 million read, most of it cached context the agents re-read, and 182,000 written. Reading 180 items costs $0.62. Cost set three choices: Haiku reads first, daily alerts are code, and live runs never batch.

HOW I WORKED

I set the direction and approved every deploy, migration and paid run. Claude Code wrote the code from a written plan, and an independent review checked every change. A separate model did all the labelling and judging.

The core went through four versions in a week. Each replay showed what to cut: too many agents, too much reading, reports that read like an audit trail.

What the agents verified

While building: tests run by hand before every deploy (code, database, browser), numbered review rounds, planted false stories. In every run: quotes, numbers, tickers, support, the edition contract, a rubric review.

What works

The weekly run end to end, daily change checks, two editions, resuming a failed run, the watchdog, and the app with feedback.

WHERE THE LOOP BROKE

01 **The filter dropped a system card.** Auditing what it threw away changed two rules.

02 **The support check was not repeatable.** A real story failed on rerun and was cut.

03 **The new version under-alerted.** The judge preferred the old alerts, so I recalibrated.

04 **No browser in the sandbox.** A screenshot script exists, but no run executes it.

09 / FUTURE WORK · PRODUCT

What I would build next.

The first four change how Wire learns and what it costs. The rest change what it does for an analyst on Monday morning.

01 GOLDEN SET

Learn from the teams

TODAY Every label so far is a model's.

NEXT Sit with each team and collect the stories they found valuable. Tune the instructions, models and scoring against that set, and score on weeks it has not seen.

A session per team, then ½ day

02 MARKET DATA

Tie stories to price moves

TODAY Wire does not see prices. DeepSeek's cost was public a month before the market moved.

NEXT Pull daily prices for every tracked listed company from a market-data API such as Alpaca, back through the history. Match each past story to what the stock did next: grade old calls, then tune the scoring for new ones.

3–4 days

03 FEEDBACK

Make feedback easy, and a game

TODAY Readers can vote and comment. Nothing reads it.

NEXT Add voice notes on any story. Keep a leaderboard and reward the most useful feedback each month. Every note feeds the golden set.

1–2 days

04 COST

Run on a subscription, not the API

TODAY Every run pays per token.

NEXT Run the same prompts, skills and checks as a Claude skill or plugin, as ChatGPT agents, or as headless Claude Code on a plan the customer already pays for. The replay already ran its judging stages this way; logging is the part to add.

2–3 days

05 PORTFOLIO

Alerts tied to positions

TODAY Alert rules do not read position sizes.

NEXT Give the alert rules the customer's positions, and alert on new evidence about any position above 5%.

About 1 day

06 DELIVERY

Push alerts to readers

TODAY Alerts wait in the app, and a story can be a week old.

NEXT Post alerts to a readers' Slack channel the day they fire, and let an urgent one start a mid-week run.

½ day

07 COVERAGE

Find sources before they matter

TODAY Sources are added by hand.

NEXT When stories keep citing a site Wire does not track, add it. Turn recurring names into trial sources, and print each week's known gaps.

2 days

08 SPLIT BY TEAM

Each team gets its own Wire

TODAY Both editions read one register and one set of prompts. A curator's switch applies to everyone; prompts change only by deploy.

NEXT When the golden set shows how the teams differ, give each its own sources and prompt: investment follows the chips, memory, hosting and power behind lab news; AI follows what is worth testing. Each team edits its part in the app, with every edit versioned. Shared facts stay in one store, so the editions still agree.

About 1½ weeks

Not next Credit risk at AI-infrastructure borrowers is the live bear case. Neither lab news nor share prices show it; that needs credit data.

10 / FUTURE WORK · ENGINEERING

What I would harden next.

The changes that take Wire from a working system to one a team can depend on, in order.

#	ENHANCEMENT	WHAT IT GIVES	EFFORT
1	**A second provider.** The same prompts, tools and checks on OpenAI's Responses API.	A run still happens when the Claude platform is down. Both run on the same weeks and are compared before the fallback is trusted.	3–5 days
2	**Narrower access for the pipeline.** Its own limited key, and the documents the analyst opens itself read by a call with no tools.	An injected page can do nothing beyond the pipeline's own tables.	1–2 days
3	**A release gate.** Both editions go out together, only after review passes and code confirms every number and date matches across them.	Readers never see one edition without the other, an unreviewed one, or two that disagree.	1½ days
4	**Stronger run guarantees.** One active run at a time, and story updates in a single transaction.	No double runs and no duplicate stories, whatever fails midway.	½ day
5	**CI on every commit.** Deploy only from a green build.	The code that runs is always the code that passed.	2–3 h
6	**Monitoring for what did not happen.** An alert when a scheduled run never starts; a tested backup and restore.	A missing edition is noticed within minutes, and data can be recovered.	1 day
7	**A visual check of each edition.** Run the existing screenshot script outside the agent sandbox, on every run.	Layout problems are caught before a reader sees them.	½ day
8	**A steadier support check.** A majority of three votes on whether a quote supports a claim.	Real stories are not cut by one unlucky answer.	2 h

The SLO I would commit to The Monday edition is out by 08:00 UTC in 11 of 12 weeks. Scheduled runs get a 2-hour ceiling instead of 4, so a failed, stalled or missing run alerts a curator by 06:30 UTC, with time for one restart before 08:00.